

How AVR Uses USART

AVR USART



MikroDuino

Amer Iqbal Qureshi | Mikrotronics Pakistan

The AVR USART Module (ATmega328P)

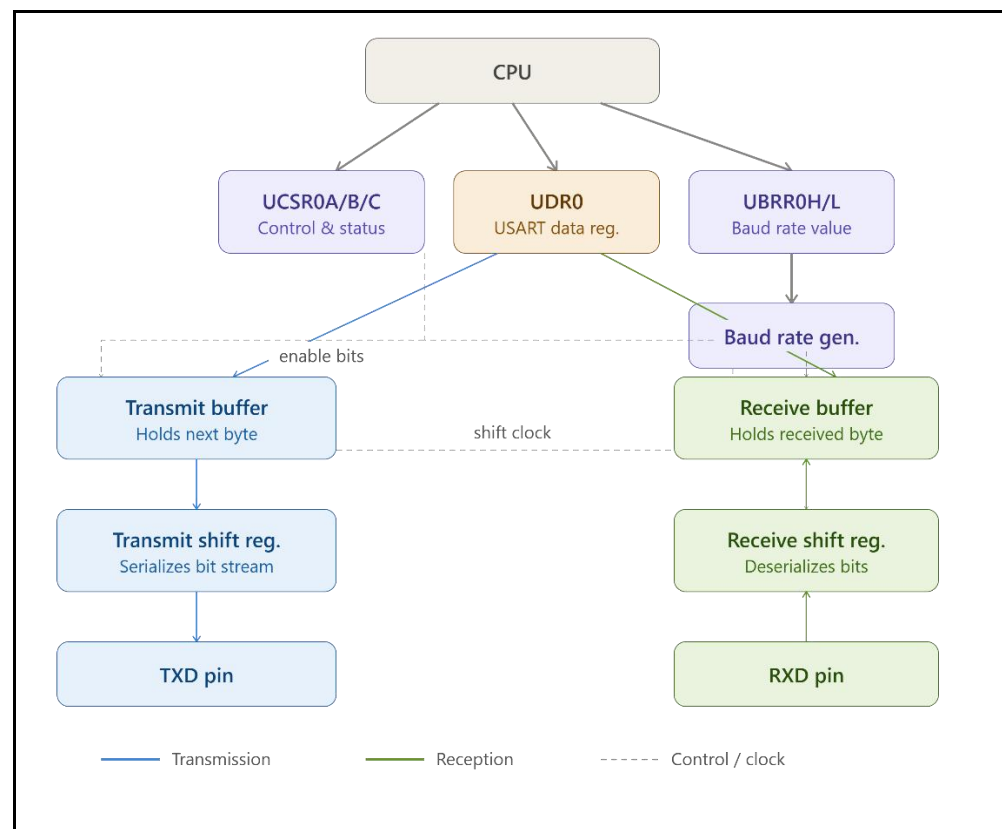
The Universal Synchronous and Asynchronous Receiver Transmitter (USART) is one of the most sophisticated peripherals inside the ATmega328P. In the previous chapter we learned how asynchronous serial communication works—start bits, stop bits, baud rate, parity, and timing. Now it is time to look inside the microcontroller itself and understand how the USART hardware performs all of those tasks automatically.

Unlike bit-banging techniques where software manually generates every pulse, the USART module contains dedicated hardware that continuously monitors the receive pin, reconstructs incoming bytes, stores them safely in hardware buffers, and transmits outgoing data with almost no CPU intervention. Once configured correctly, the processor simply places data into the transmitter or reads data from the receiver while the USART handles all timing with remarkable precision.

Understanding the USART registers is important because every Arduino Serial function, every MikroDuino serial class, and almost every embedded communication library eventually manipulates these very same hardware registers.

Architecture of the AVR USART

The USART peripheral may be imagined as two independent engines working simultaneously. One engine continuously transmits data. The other continuously receives data. Both share the baud rate generator and communicate with the CPU through data registers.



USART Registers

The ATmega328P USART is controlled through several registers.

Register	Purpose
UDR0	USART Data Register
UCSR0A	Status Register
UCSR0B	Control Register B
UCSR0C	Control Register C
UBRR0H	Baud Rate High Byte
UBRR0L	Baud Rate Low Byte

Think of these as the dashboard of the USART. Some registers tell the hardware what to do. Others tell the CPU what the hardware is currently doing.

Baud Rate Registers (UBRR0H and UBRR0L)

The USART must know exactly how fast to send bits. This is achieved using a programmable divider called the Baud Rate Generator. The divisor is stored as a 12-bit number split into two registers.

```
UBRR0H
+-----+
| 11....8 |
+-----+

UBRR0L
+-----+
| 7.....0 |
+-----+
```

Together they form

```
UBRR = (UBRR0H << 8) | UBRR0L
```

The baud rate is calculated as

$$[\text{Baud} = \frac{F_{\text{CPU}}}{16(\text{UBRR}+1)}]$$

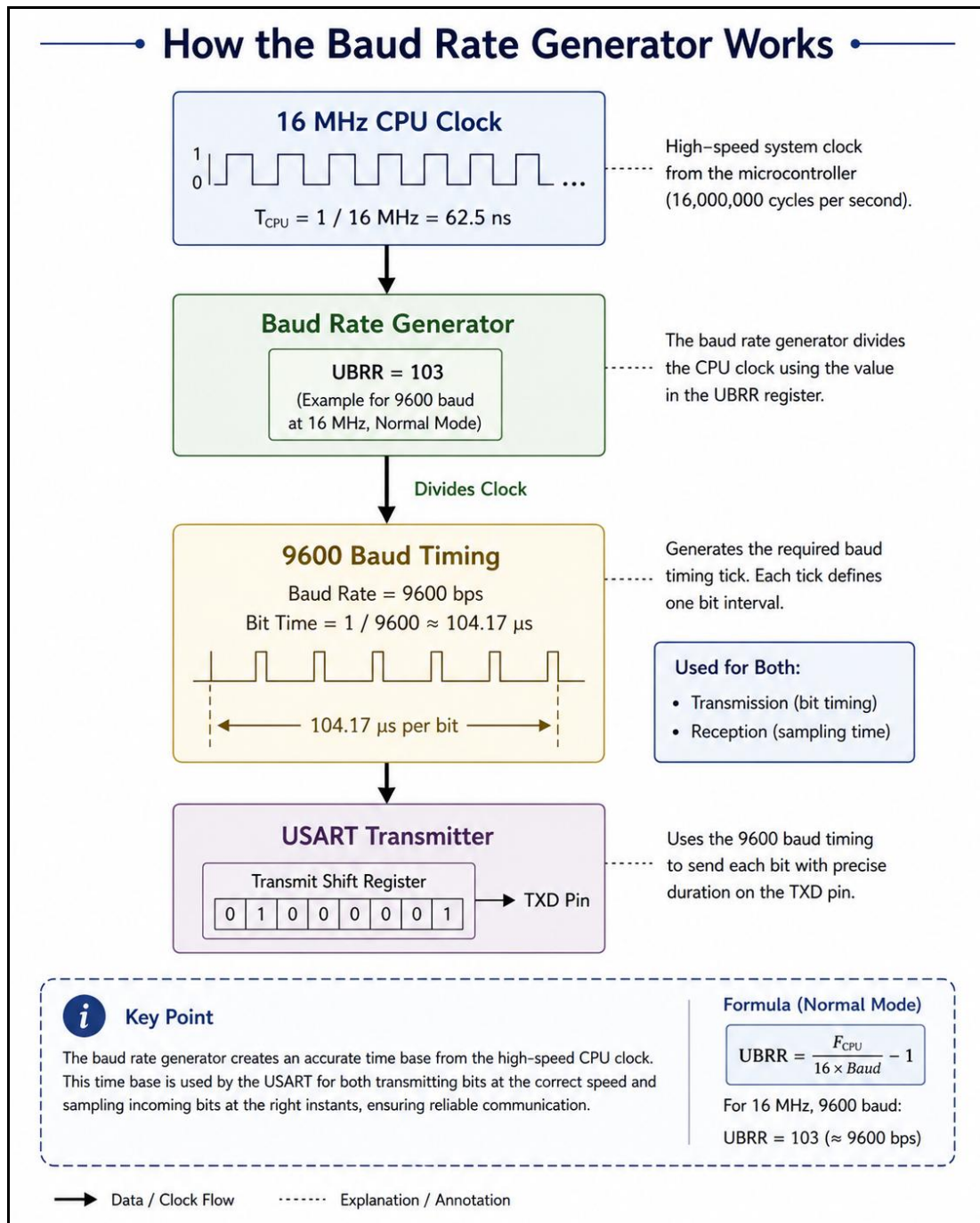
For a 16 MHz clock,

9600 baud gives

```
UBRR = 103
```

which becomes

```
UBRR0H = 0
UBRR0L = 103
```



The USART Data Register (UDR0)

This is the register the programmer interacts with most often. Whenever data is transmitted,

```
UDR0 = 'A';
```

the CPU writes into UDR0. Whenever data is received,

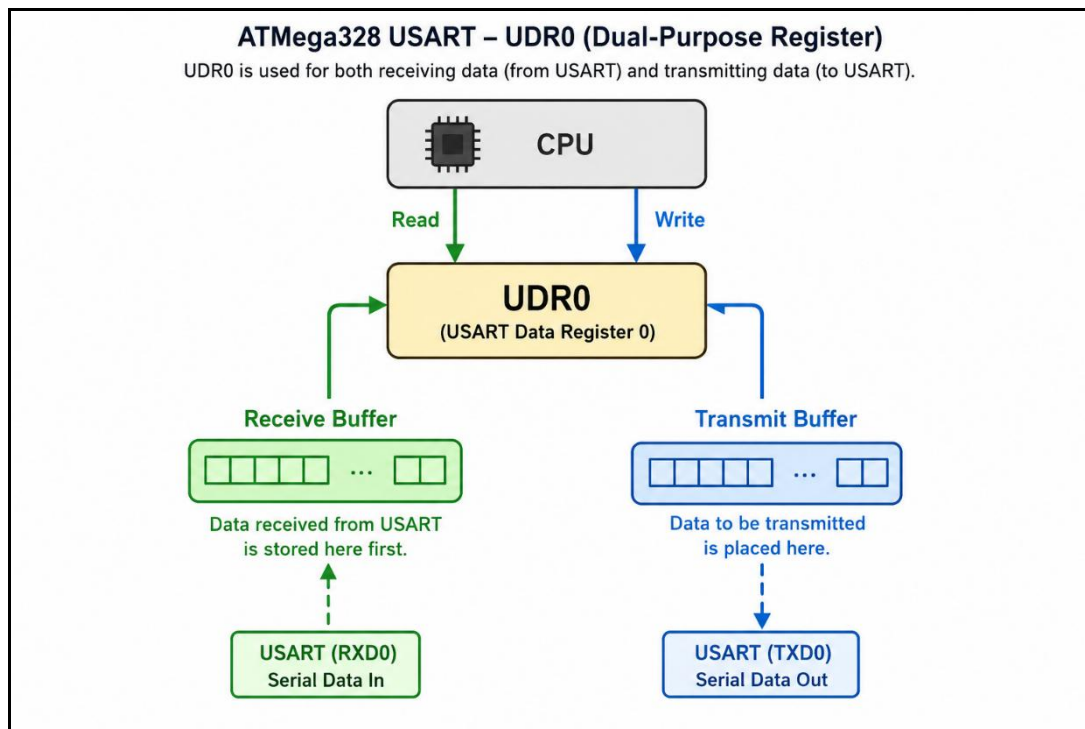
```
char c = UDR0;
```

the CPU reads from UDR0.

Although it appears to be one register, internally it behaves as two independent registers:

- transmit buffer
- receive buffer

The hardware automatically decides which one is accessed depending upon whether the CPU is reading or writing.

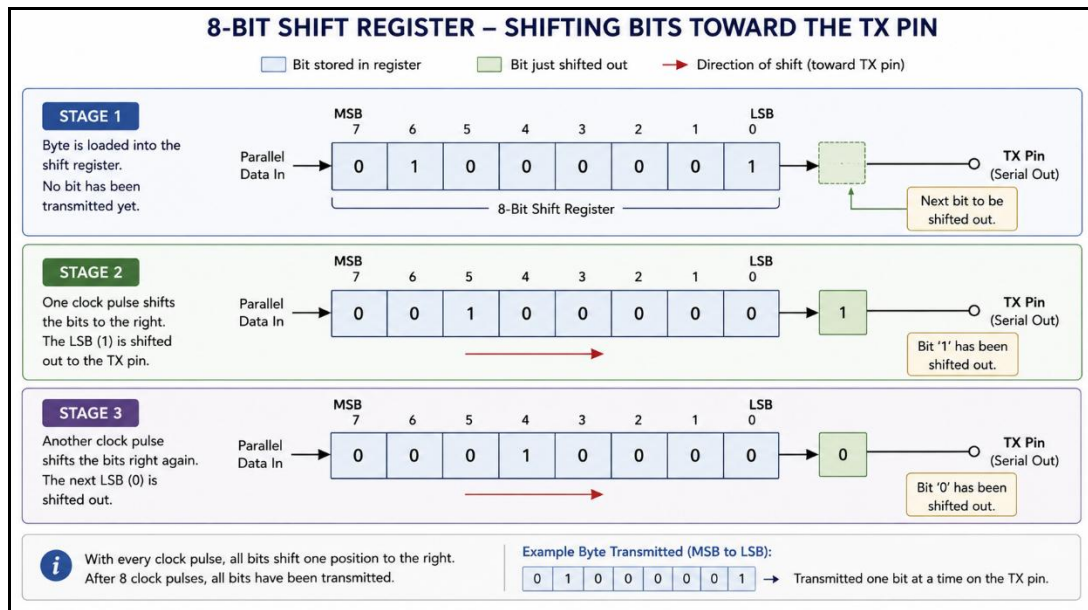


The Transmit Buffer

When software writes a byte into UDR0, the byte first enters the transmit buffer.

```
UDR0 = 'H';
```

The hardware waits until the shift register becomes empty. Then it automatically copies the byte into the transmit shift register. The CPU is immediately free to perform other tasks.



The Receive Buffer

Reception works in reverse. Incoming serial bits are assembled inside the receive shift register. After the stop bit has been received successfully, the byte moves into the receive buffer.

The CPU later reads

UDR0

to obtain the completed byte.

Double Buffering

One particularly clever feature of the AVR USART is double buffering. Suppose the CPU has not yet read the previous byte. Meanwhile another byte arrives. Instead of losing it immediately, the hardware temporarily stores the second byte while the first one remains available for the CPU.

This greatly reduces the chances of losing incoming characters. However, if a third byte arrives before software has removed the earlier bytes, the receiver can no longer store it. A Data OverRun error occurs.

Data OverRun

This is one of the most common USART errors. The receiver is working correctly. The sender is transmitting correctly. The problem is that the CPU is too slow. It failed to remove previous data before additional characters arrived.

Imagine a receptionist receiving telephone calls. If one caller is already waiting, and another is on hold, a third caller cannot be accepted.

Exactly the same thing happens inside the USART. The DOR0 flag inside UCSR0A is automatically set. Software should monitor this flag whenever reliable communication is required.

UCSR0A — Status Register

This register contains information about the current state of the USART.

Important bits include:

Bit	Name	Purpose
RXC0	Receive Complete	
TXC0	Transmission Complete	
UDRE0	Data Register Empty	
FE0	Framing Error	
DOR0	Data OverRun	
UPE0	Parity Error	

The three error bits directly correspond to the communication problems discussed in the previous chapter.

- Missing stop bit → FE0
- Buffer full → DOR0
- Wrong parity → UPE0

This demonstrates how the hardware detects communication faults and informs software.

UCSR0A – USART Control and Status Register A (8-bit)								
Bit	7	6	5	4	3	2	1	0
UCSR0A	RXC0 0	TXC0 0	DOR0 0	FE0 0	UPE0 0	UDRE0 0	U2X0 0	MPCM0 0

7	RXC0	Receive Complete Set to 1 when a character is received and transferred to UDR0. Cleared by reading UDR0.
6	TXC0	Transmit Complete Set to 1 when the entire frame (including stop bit) has been transmitted. Cleared by writing a logic 1 to this bit.
5	DOR0	Data OverRun Set to 1 when a new byte is received before the previous byte in UDR0 was read. Cleared by reading UDR0.
4	FE0	Frame Error Set to 1 when a framing error is detected (no stop bit). Cleared by reading UDR0.
3	UPE0	Parity Error Set to 1 when a parity error is detected. Cleared by reading UDR0.
2	UDRE0	USART Data Register Empty Set to 1 when UDR0 is ready to accept a new byte for transmission. Cleared by writing data to UDR0.
1	U2X0	Double the USART Transmission Speed 0 = Normal speed, 1 = Double speed.
0	MPCM0	Multi-processor Communication Mode 0 = Disabled, 1 = Enabled (address mark wake-up).

UCSR0B — Control Register

This register enables the transmitter and receiver.

Important bits include

Bit	Function
RXEN0	Enable Receiver
TXEN0	Enable Transmitter
RXCIE0	Receive Interrupt Enable
TXCIE0	Transmission Complete Interrupt
UDRIE0	Data Register Empty Interrupt

Without setting RXEN0 and TXEN0, the USART remains disabled.

UCSR0C — Frame Format Register

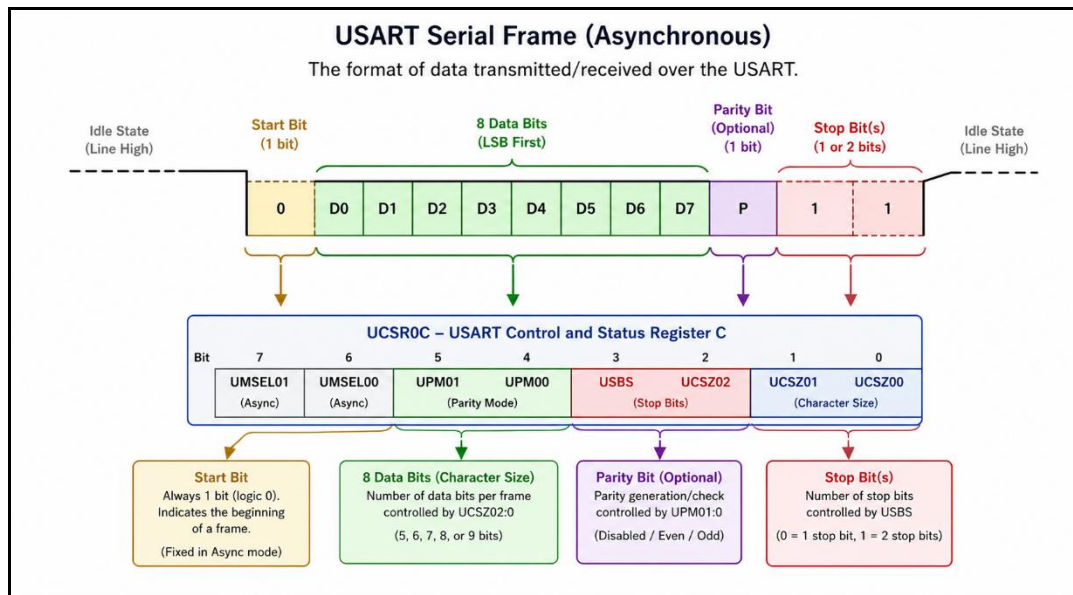
This register determines how each frame looks.

It controls

- Number of data bits
- Stop bits
- Parity
- Asynchronous/Synchronous mode

Typical Arduino configuration:

```
8 Data Bits  
No Parity  
1 Stop Bit  
(8N1)
```

Initializing the USART

A minimal initialization sequence consists of:

```
#define F_CPU 16000000UL

#include <avr/io.h>

void USART_Init()
{
    uint16_t ubrr = 103;

    UBRR0H = ubrr >> 8;

    UBRR0L = ubrr;

    UCSR0B = (1 << RXEN0) | (1 << TXEN0);

    UCSR0C = (1 << UCSZ01) | (1 << UCSZ00);
}
```

This configures:

- 9600 baud
- 8 data bits
- No parity
- One stop bit
- Receiver enabled
- Transmitter enabled

Sending One Character

```
void USART_SendChar(char c)
{
    while (!(UCSR0A & (1 << UDRE0)));

    UDR0 = c;
}
```

The loop waits until the transmit buffer becomes empty.

Only then is the next character written.

Receiving One Character

```
char USART_ReadChar()
{
    while (!(UCSR0A & (1 << RXC0)));

    return UDR0;
}
```

The function waits until a complete byte has arrived. The hardware then transfers the received byte from the receive buffer into the CPU.

Sending a String

```
void USART_SendString(const char *str)
{
    while (*str)
    {
        USART_SendChar(*str++);
    }
}
```

Usage

```
USART_SendString("Hello World\r\n");
```

Echo Program

One of the simplest demonstrations is an echo application. Every received character is immediately transmitted back.

```
#include <avr/io.h>

int main()
{
    USART_Init();
```

```

while (1)
{
    char c = USART_ReadChar();

    USART_SendChar(c);
}

```

Typing

Hello

on a serial terminal immediately returns

Hello

Converting Characters to Uppercase

```

int main()
{
    USART_Init();

    while (1)
    {
        char c = USART_ReadChar();

        if (c >= 'a' && c <= 'z')
            c -= 32;

        USART_SendChar(c);
    }
}

```

Typing

mikroduino

returns

MIKRODUINO

Sending Numbers

```

#include <stdlib.h>

char buffer[10];

int main()
{
    USART_Init();

    int counter = 0;

    while (1)
    {

```

```
    itoa(counter++, buffer, 10);

    USART_SendString(buffer);
    USART_SendString("\r\n");

    for (volatile long i = 0; i < 50000; i++);
}
}
```

The terminal displays an increasing sequence of integers, demonstrating how numeric values can be converted to text before transmission.

Summary

The USART module of the ATmega328P is far more than a simple serial pin. It is a complete hardware communication engine containing a baud rate generator, transmit and receive shift registers, double-buffered data storage, configurable frame formatting, status reporting, and error detection. By using registers such as **UBRR0H/L**, **UCSR0A**, **UCSR0B**, **UCSR0C**, and **UDR0**, software can communicate efficiently with external devices while the hardware manages the precise timing of serial communication. These low-level techniques form the foundation upon which higher-level interfaces such as Arduino's `Serial` class and the MikroDuino SDK are built, making them essential knowledge for anyone wishing to master embedded systems programming.